

SELECT

-- [Step 4] Finally, we select and calculate the columns to display.

Warehouse.warehouse_id,

-- [Step 4] Combine state and alias into a readable name.

CONCAT(Warehouse.state, ': ', Warehouse.warehouse_alias) AS warehouse_name,

-- [Step 4] Count how many orders belong to this specific warehouse group.

COUNT(Orders.order_id) AS number_of_orders,

-- [Step 4] SUBQUERY: Calculate the total orders across ALL warehouses.

-- Note: This runs independently to get a single number (e.g., 10,000) for the math below.

(SELECT COUNT(*) FROM warehouse_orders.orders AS Orders) AS total_orders,

-- [Step 4] LOGIC: Compare this warehouse's count vs. the total count calculated above.

CASE

WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM warehouse_orders.orders AS Orders) <= 0.20

THEN 'Fulfilled 0-20% of Orders'

WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM warehouse_orders.orders AS Orders) > 0.20

AND COUNT(Orders.order_id)/(SELECT COUNT(*) FROM warehouse_orders.orders AS Orders) <= 0.60

THEN 'Fulfilled 21-60% of Orders'

ELSE 'Fulfilled more than 60% of Orders'

END AS fulfillment_summary

FROM

-- [Step 1] START HERE: Pick the primary table (Warehouses).

-- "LEFT JOIN" means to keep all warehouses, even if they have no orders.

-- "LEFT JOIN" is protected.

warehouse_orders.warehouse AS Warehouse -- This is the left table

LEFT JOIN

-- [Step 1] Merge it with the Orders table.

-- This is the "RIGHT TABLE" which is unprotected

warehouse_orders.orders AS Orders

ON Orders.warehouse_id = Warehouse.warehouse_id

GROUP BY

```
-- [Step 2] Collapse the thousands of rows into groups, one per warehouse.  
Warehouse.warehouse_id,  
warehouse_name
```

HAVING

```
-- [Step 3] Filter the groups. Only keep warehouses that actually have orders.  
-- This removes any "empty" warehouses that came in from the LEFT JOIN.  
COUNT(Orders.order_id) > 0
```